0416. Partition Equal Subset Sum

gory: 1-D Dynamic Programming | LeetCode: https://leetcode.com/problems/part

Problem

- Can the array be split into two subsets with equal sums?

Intuition

- Equivalent to: is there a subset summing to total / 2? (total must be even.)
- 0/1 knapsack on a boolean array: can[s] = can we reach sum s with the items so far.

Brute Force

- Try all subsets: O(2^n).

Optimal Approach

- If total is odd return false; target = total / 2.
- Can[0] = true. For each x, for s from target down to x: can[s] |= can[s - x].
- Return can[target].

Worked Example

```
[1,5,11,5], target 11.

x=1  reach {0,1}
x=5  reach {0,1,5,6}
x=11 reach {0,1,5,6,11,...}  -> 11 reachable -> true
```

Java Solution

```java
public class PartitionEqualSubsetSum {
    public boolean canPartition(int[] nums) {
        int total = 0;
        for (int x : nums) total += x;
        if (total % 2 == 1) return false;
        int target = total / 2;
        boolean[] can = new boolean[target + 1];
        can[0] = true;
        for (int x : nums)
            for (int s = target; s >= x; s--) can[s] |= can[s - x];
        return can[target];
    }
}
```

Complexity

- Time: O(n target). Space: O(target).

Pitfalls

- Iterate s downwards so each item is used at most once.